

PROJECT VERACITY — COMPLETE CONTEXT FILE

READ THIS FIRST. This file is the single source of truth for Project Veracity. Paste this entire file into Claude (or any AI) before asking for help with this project. It contains everything: hackathon context, architecture, tech stack, file structure, build order, code patterns, and performance targets.

HACKATHON DETAILS

Field	Value
Event	XEN-O-THON 2026
Organizer	GTBIT, New Delhi — Institution's Innovation Council (Ministry of Education)
Team Name	Leo
Track	AI & Automation (Beyond Wrappers)
Project Title	Project Veracity — The Self-Healing Hallucination Firewall
Category	Software / AI Infrastructure
Demo Domain	Financial Analysis Assistant

PROBLEM STATEMENT (OFFICIAL)

The Three Pain Points

1. **Enterprise Trust Deficit** — A single hallucinated financial metric or fake legal precedent can cost millions in lawsuits and reputation damage. Enterprise AI is stuck in the "toy" phase.
2. **The Latency Trap** — Current solutions wait for full LLM generation, then run a secondary verification prompt. This destroys the real-time streaming UX.
3. **The "Useless Flag" Problem** — Simply warning "Potentially Incorrect" leaves users frustrated. They want the correct answer, not a warning.

Official One-Line Deliverable

"Build a low-latency, real-time AI firewall that intercepts streaming LLM outputs, fact-checks claims against a local ground-truth database, and auto-corrects hallucinations before they reach the user's screen."

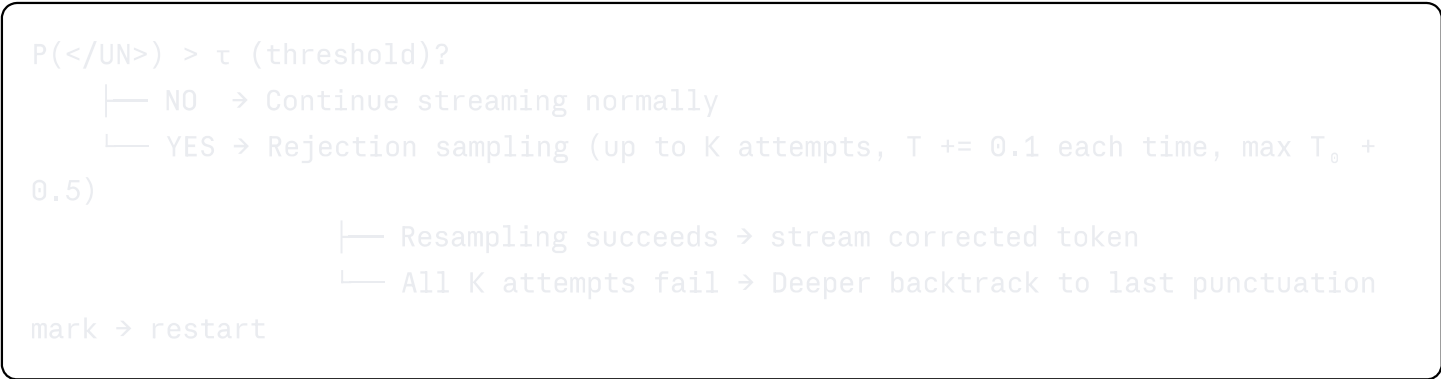
Three Hard Technical Requirements (Must-Haves)

- 1. **Streaming Interceptor** — Process SSE token stream in real-time. Detect claim boundaries (numbers, names, policies) on the fly without adding massive latency.
- 2. **Reverse-RAG Verification Engine** — Build a local Vector Database (Ground Truth Vault). As the LLM streams a claim, instantly perform semantic search to verify factual accuracy.
- 3. **Self-Healing Auto-Correct UI** — Intercept the hallucination, retrieve the real fact from Vector DB, and seamlessly rewrite the sentence. Frontend must visually highlight the fix in green with tooltip: [Auto-Corrected via Verified Source: Document_Name.pdf]

🏗️ SYSTEM ARCHITECTURE



REVERSE Algorithm Decision Flow



 TECH STACK (COMPLETE)

Backend Core

Component	Library/Framework	Key Syntax / Functions	Purpose
Web Framework	FastAPI	<code>@app.get("/stream")</code> , <code>StreamingResponse(generator, media_type="text/event-stream")</code>	SSE endpoint creation
Async Programming	asyncio (built-in)	<code>async def</code> , <code>await</code> , <code>async for</code> , <code>asyncio.gather()</code>	Non-blocking I/O for streaming
Data Validation	Pydantic	<code>class Claim(BaseModel): text: str</code> , <code>Field()</code>	Request/response schemas
Server	uvicorn	<code>uvicorn main:app --reload</code>	ASGI server for FastAPI

AI / ML

Component	Library/Framework	Key Syntax / Functions
NLI Model	transformers (HuggingFace)	<code>AutoModelForSequenceClassification.from_pretrained()</code> , <code>AutoTokenizer</code>
Deep Learning	torch (PyTorch)	<code>torch.no_grad()</code> , <code>torch.softmax()</code> , <code>model(**inputs)</code>

Component	Library/Framework	Key Syntax / Functions
Embeddings	sentence-transformers	<code>SentenceTransformer("all-MiniLM-L6-v2")</code> , <code>model.encode()</code>

Vector Database

Component	Library	Key Syntax	Purpose
Primary DB	chromadb	<code>PersistentClient()</code> , <code>collection.add()</code> , <code>collection.query()</code>	Reverse-RAG Vault storage
Alternative	faiss (optional)	<code>faiss.IndexFlatIP()</code> , <code>index.add()</code> , <code>index.search()</code>	Facebook AI similarity search fallback

Text Processing

Component	Library	Key Syntax	Purpose
Pattern Matching	re (built-in)	<code>re.finditer(pattern, text)</code> , <code>re.compile()</code> , lookahead/lookbehind	Claim boundary detection
String Similarity	difflib (built-in)	<code>SequenceMatcher()</code> , <code>get_close_matches()</code>	Fuzzy matching for corrections

Data & HTTP

Component	Library	Key Syntax	Purpose
Arrays	numpy	<code>np.array()</code> , <code>np.dot()</code> , cosine similarity calc	Numerical operations
JSON	json (built-in)	<code>json.dumps()</code> , <code>json.loads()</code>	SSE message formatting
HTTP Client	httpx (async)	<code>httpx.AsyncClient()</code> , <code>client.post()</code> , streaming response	Calling OpenAI/Ollama APIs

Frontend

Component	Library	Key Syntax	Purpose
Framework	Next.js 14+	<code>app/page.tsx</code> , <code>app/api/route.ts</code> , file-based routing	Full-stack React framework
UI Library	React 18+	<code>useState</code> , <code>useEffect</code> , <code>useRef</code> , JSX syntax	Component architecture
Streaming	EventSource (Browser API)	<code>new EventSource('/api/stream')</code> , <code>.onmessage</code> , <code>.onerror</code>	SSE consumption
Styling	Tailwind CSS	<code>className="bg-green-100 text-green-800 transition-all duration-300"</code>	Visual correction theatre
Types	TypeScript	<code>interface Token { id: string; text: string; status: 'pending' or 'verified' or 'corrected' }</code>	Type safety

DevOps / Testing

Component	Library	Key Syntax	Purpose
Environment	python-dotenv	<code>load_dotenv()</code> , <code>os.getenv("OPENAI_KEY")</code>	Config management
Process Mgmt	PM2 or systemd	<code>pm2 start server.py --interpreter python3</code>	Production process control
Testing	pytest, httpx	<code>client = TestClient(app)</code> , <code>async def test_stream()</code>	Endpoint validation
Logging	loguru or logging	<code>logger.add("file.log")</code> , structured logging	Pipeline observability

 INSTALLATION COMMANDS

bash

```

# — BACKEND —————
cd project-veracity/backend

# Create and activate virtual environment (Windows PowerShell)
python -m venv .venv
.venv\Scripts\Activate.ps1

# If activation is blocked by policy, run this first:
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser

# Install all Python dependencies
pip install fastapi uvicorn[standard] transformers torch chromadb sentence-transformers

# Install dev/testing tools
pip install pytest httpx pytest-asyncio

# — FRONTEND —————
cd ../frontend

# Initialize Next.js project (answer prompts: React Compiler=Yes, Import Alias=press
npx create-next-app@latest veracity-ui --typescript --tailwind --eslint --app --src-

# Install dependencies
cd veracity-ui && npm install

```

📁 COMPLETE FILE STRUCTURE

```

project-veracity/
├──
├── backend/
│   ├── .venv/                ← Python virtual environment (DO NOT EDIT)
│   ├── vault_db/            ← ChromaDB persistent storage (auto-
created at runtime)
│   ├── .env                 ← API keys: OPENAI_API_KEY=sk-...
│   ├── models.py            ← Pydantic schemas: Claim, Token,
VerificationResult, SSEEvent
│   ├── vault.py              ← ChromaDB operations: initialize,
add_document, search
│   ├── sentinel.py           ← HaluGate NLI wrapper: load model,
classify claim vs context

```

└─ rewriter.py	← REVERSE algorithm: backtrack + rewrite
hallucinated sentences	
└─ interceptor.py	← SSE stream handler + regex claim
boundary detector	
└─ main.py	← FastAPI app entry: wire all modules,
expose /stream endpoint	
└─ frontend/	
└─ veracity-ui/	
└─ public/	← Static assets (auto-generated by
Next.js)	
└─ file.svg	
└─ globe.svg	
└─ next.svg	
└─ vercel.svg	
└─ window.svg	
└─ src/	
└─ app/	
└─ favicon.ico	
└─ globals.css	← Tailwind base styles
└─ layout.tsx	← Root layout (auto-generated)
└─ page.tsx	← MAIN UI: SSE consumer + correction
theatre ← BUILD THIS	
└─ components/	← CREATE THIS FOLDER
└─ TokenStream.tsx	← Token display: green highlight + tooltip
on corrections	
└─ types/	← CREATE THIS FOLDER
└─ index.ts	← TypeScript interfaces: Token,
StreamEvent, SessionStats	
└─ .gitignore	
└─ eslint.config.mjs	
└─ next.config.ts	
└─ package.json	
└─ package-lock.json	
└─ postcss.config.mjs	
└─ README.md	
└─ tsconfig.json	

BUILD ORDER (STRICT — Follow This Sequence)

Step	Order	Focus	Language	Primary Library	Goal
1	1st	Async Basics	Python	asyncio	Understand async/await patterns
2	2nd	Web Framework	Python	FastAPI	Create basic HTTP endpoints
3	3rd	SSE Implementation	Python	FastAPI StreamingResponse	Stream tokens to browser
4	4th	ML Model Loading	Python	transformers, torch	Load NLI model, run inference
5	5th	NLI Classification	Python	transformers	Classify entailment/contradiction
6	6th	Vector Database	Python	chromadb	Store and retrieve embeddings
7	7th	Claim Detection	Python	re (regex)	Detect numbers, names, dates in stream
8	8th	Integration	Python	All backend libs	Connect detection → verification → correction
9	9th	Frontend Setup	TypeScript	Next.js	Initialize project, configure Tailwind
10	10th	SSE Consumption	TypeScript	EventSource (native)	Receive and display tokens
11	11th	State Management	TypeScript	React hooks	Update UI on corrections
12	12th	Visual Polish	TypeScript/CSS	Tailwind CSS	Animations, color coding, stats panel
13	13th	End-to-End	Both	Full stack	Demo-ready pipeline

File Build Order

```
models.py → vault.py → sentinel.py → rewriter.py → interceptor.py → main.py  
→ types/index.ts → components/TokenStream.tsx → app/page.tsx  
→ End-to-End Test
```

🔗 CRITICAL SYNTAX PATTERNS

Python — SSE Generator (core of the firewall)

```
python  
  
yield f"data: {json.dumps(data)}\n\n"
```

Python — Async token processing loop

```
python  
  
async for token in llm_stream:  
    # detect → verify → yield
```

Python — Parallel claim verification (non-blocking)

```
python  
  
await asyncio.gather(*[verify(c) for c in claims])
```

Python — Pydantic model definition

```
python  
  
class Claim(BaseModel):  
    text: str  
    type: str
```

Python — NLI inference (no gradient)

```
python  
  
with torch.no_grad():  
    outputs = model(**inputs)
```

Python — ChromaDB vault search

```
python

collection.query(query_texts=[claim], n_results=3)
```

TypeScript — React state for tokens

```
typescript

const [tokens, setTokens] = useState<Token[]>([])
```

TypeScript — SSE connection with useEffect

```
typescript

useEffect(() => {
  const es = new EventSource('/api/stream')
  es.onmessage = (e) => { const data = JSON.parse(e.data) }
}, [])
```

TypeScript — Conditional Tailwind class for correction highlight

```
typescript

className={`token ${status === 'corrected' ? 'bg-green-100 text-green-800' : ''}`}
```

⚡ PERFORMANCE TARGETS

Metric	Target	Technology	Optimization
NLI Inference	<12ms	ModernBERT-base, torch.no_grad()	Quantization, CPU inference
Vector Search	<100ms	ChromaDB HNSW index	Local embedding, caching
Total Pipeline	<200ms	asyncio parallel tasks	Non-blocking architecture
Token Latency	<50ms	SSE streaming	No buffering, immediate yield
Frontend Render	<16ms	React virtual DOM	Efficient state updates

Competitive Benchmark

Approach	Latency	Self-Healing	Efficiency Gain
LLM-as-Judge (GPT-4)	2,000-5,000ms	No	0%
Post-generation RAG checkers	500-1,000ms	No	0%
Project Veracity (Ours)	<200ms	YES	72.2%

🧠 KEY ALGORITHMS & REFERENCES

HaluGate Sentinel

- ModernBERT-based NLI classifier
- 96.4% validation accuracy on fact-seeking detection
- 12ms inference latency
- 72.2% compute efficiency gain by skipping non-factual queries (~35% of all queries)
- NLI labels: Entailment / Neutral / Contradiction
- Cold start: <500ms (Rust/Candle native integration in original paper)

REVERSE Algorithm

- Uses ungrounded token $P(</UN>)$ probability to detect hallucinations
- Thresholds: $\tau = 0.003$ (generative tasks), $\tau = 0.5$ (discriminative tasks)
- On $P(</UN>) > \tau$: Rejection sampling up to K times
- Temperature adjustment: $T = \min(T + 0.1, T_0 + 0.5)$ per resample attempt
- If all K samples fail: Deeper backtrack to last punctuation mark → restart generation

Kernel Language Entropy (Detection Reference)

- Von Neumann entropy of density matrix: $S(\rho) = -\text{Tr}(\rho \ln \rho)$
- Used for uncertainty estimation across multiple LLM response samples

VIGIL (Optional Future Integration)

- Self-healing runtime with affective state tracking (EmoBank model)
- Exponential decay: $I_{\text{decayed}} = I_{\text{initial}} \times 0.5^{((t'-t)/h)}$ where $h = 12\text{-hour half-life}$

- RBT schema: Roses (successes) / Buds (opportunities) / Thorns (critical failures trigger fixes)

ENVIRONMENT

Field	Value
OS	Windows 11 (PowerShell)
Python Version	3.14
Virtual Env Tool	python -m venv (NOT uv — uv not in PATH)
Virtual Env Location	/backend/.venv
Node Version	18+
Frontend Location	/frontend/veracity-ui/
React Compiler	Enabled (Yes was selected during init)
Import Alias	@/* (default)
Backend Port	<u>http://localhost:8000</u>
Frontend Port	<u>http://localhost:3000</u>

Starting the Servers

```
bash

# Backend (from /backend with .venv activated)
uvicorn main:app --reload --port 8000

# Frontend (from /frontend/veracity-ui)
npm run dev
```

DEMO SCENARIO

User asks: "What was Apple's revenue in fiscal year 2022?"

LLM streams (hallucinated): "Apple reported revenue of \$523 billion in fiscal year 2022..."

Firewall pipeline:

- 1. Interceptor detects: (\$523 billion) (number + entity claim)
- 2. Sentinel: IS fact-seeking → proceed
- 3. Vault search: finds (Apple Inc reported revenue of \$394.3 billion in fiscal year 2022) from (apple_annual_report.pdf)
- 4. NLI: CONTRADICTION detected (high confidence)
- 5. REVERSE rewrites: (\$523 billion) → (\$394.3 billion)
- 6. UI streams corrected text with green highlight
- 7. Tooltip: ([Auto-Corrected via Verified Source: apple_annual_report.pdf])

 CURRENT BUILD STATUS

Component	Status
Backend folder created	Done
Python .venv created + activated	Done
Frontend Next.js initialized (veracity-ui)	Done
models.py	NOT BUILT YET — build this first
vault.py	NOT BUILT YET
sentinel.py	NOT BUILT YET
rewriter.py	NOT BUILT YET
interceptor.py	NOT BUILT YET
main.py	NOT BUILT YET
types/index.ts	NOT BUILT YET
components/TokenStream.tsx	NOT BUILT YET
app/page.tsx	NOT BUILT YET

INNOVATION HIGHLIGHTS (For Jury)

- First mid-stream, sentence-level hallucination rewriter — not a checker, a healer
- Dual-stage NLI gate: Entailment / Neutral / Contradiction severity scoring per claim
- HaluGate Sentinel: 12ms classifier skips 35% of non-factual queries (72.2% compute saved)
- REVERSE backtracking: resamples K times before deeper rollback to last punctuation mark
- Domain-agnostic vault: swap in any PDF/document set; works for finance, legal, medical instantly
- Live correction theatre: users see the AI being corrected in real-time, building trust visually
- Benchmarked against: HaluEval, TruthfulQA, RAGTruth, MedHallu (10,000 medical QA pairs)